

Seq2Seq and Attention Mechanism

Rasha Obeidat, PhD

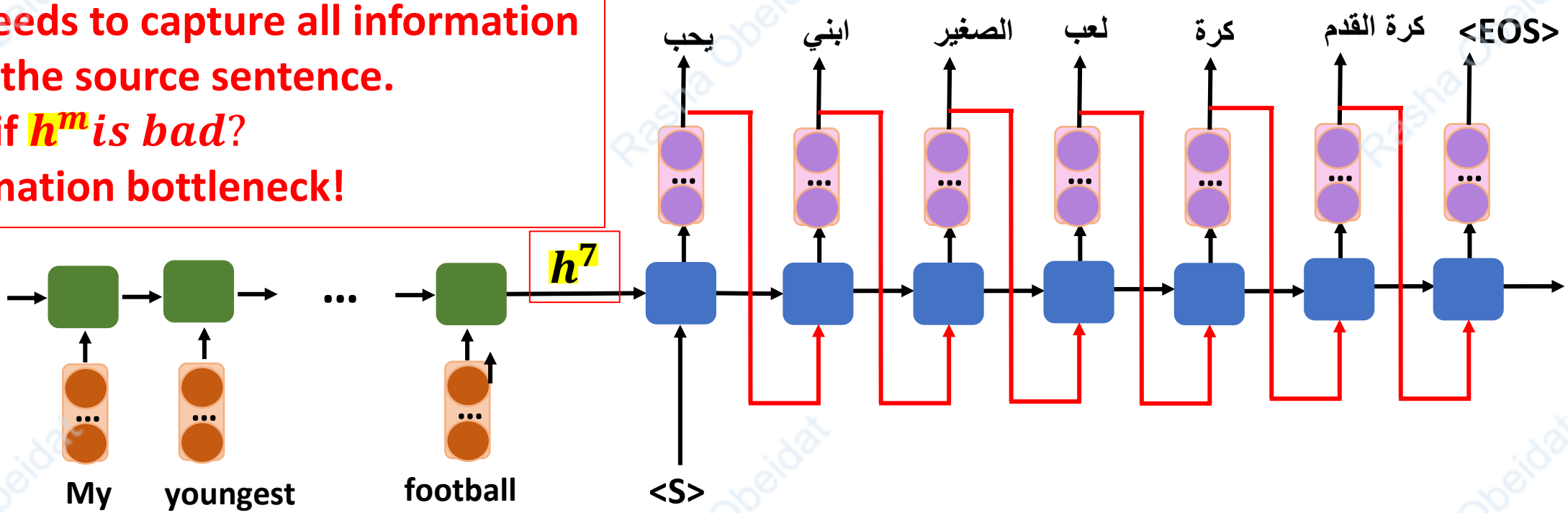
NLP

Rasha Obeidat

Seq2Seq: bottleneck problem

- What if the encoder makes a bad summary (bad context vector h^m)
  → the translation will be bad.

Encoding of the source sentence.
This needs to capture all information about the source sentence.
What if h^m is bad?
Information bottleneck!



Attention

- Solution?
- Attention mechanism that allows neural networks to focus on memorizing the important parts of input.
- The attention mechanism emerged as an improvement over the encoder decoder-based MT. Later, it was applied to a variety of tasks and in all of the fields including NLP, computer vision, speech processing, etc.

Rasha Obeidat

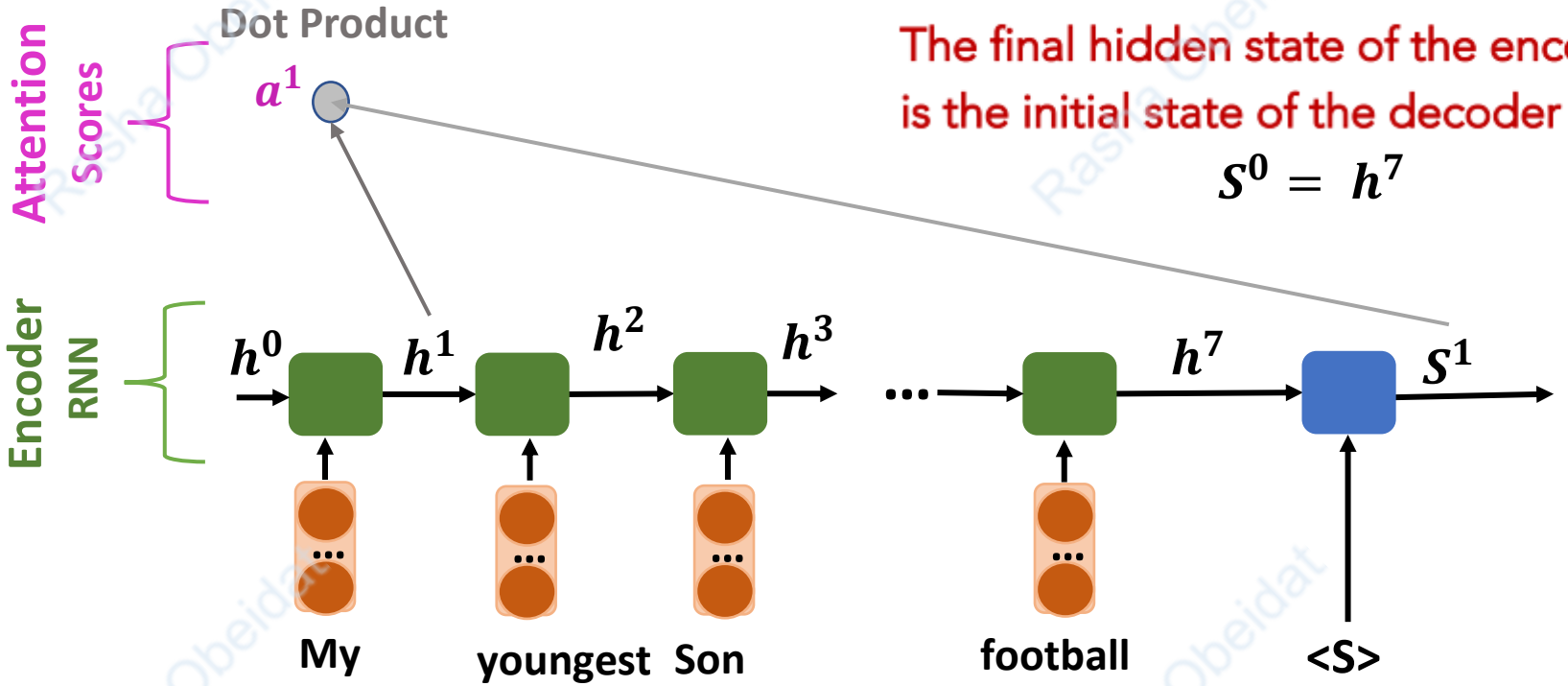
Attention

1. At each time step j in the decoding stage , for all steps of the encoder $i = 1, 2, \dots, m$

- Compute $a^{j,i} = f\left(h^i, S^j\right)$
- $h^i \in \mathbb{R}^h, S^j \in \mathbb{R}^h$

Using Dot Attention

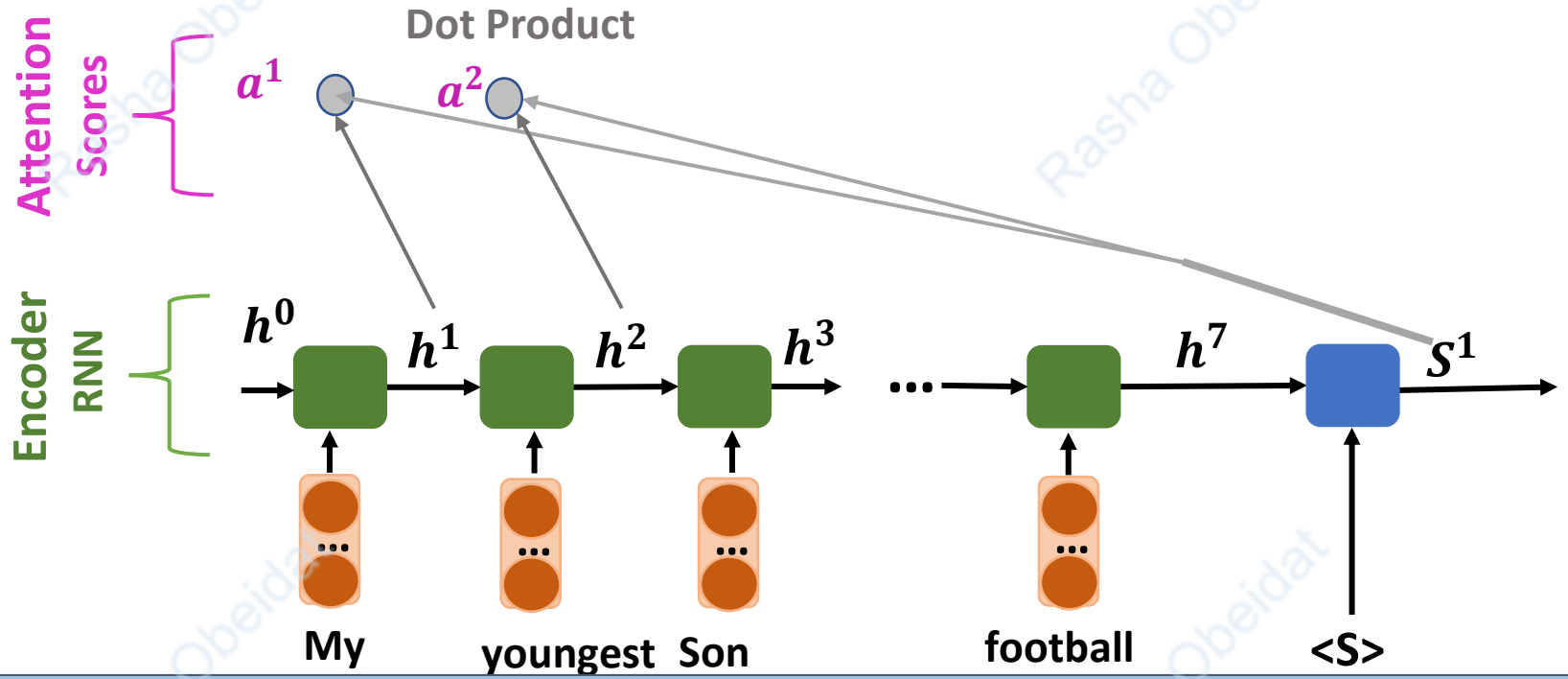
$$f\left(h^{j,i}, S^j\right) = h^{j,i^T} S^j$$



Attention

1. At each time step j in the decoding stage , for all steps of the encoder $i = 1, 2, \dots, m$

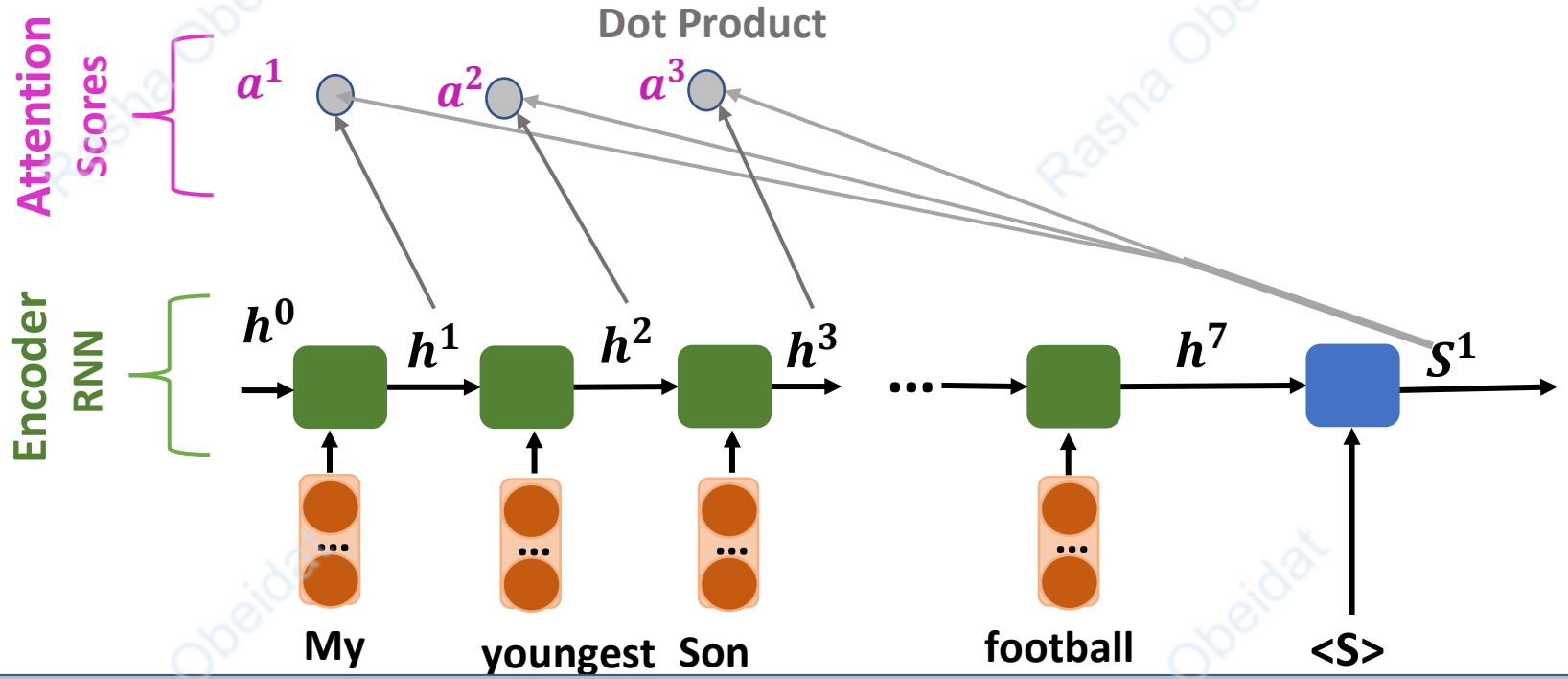
- Compute $a^{j,i} = f\left(h^i, S^j\right)$
- $h^i \in \mathbb{R}^h, S^j \in \mathbb{R}^h$



Attention

1. At each time step j in the decoding stage , for all steps of the encoder $i = 1, 2, \dots, m$

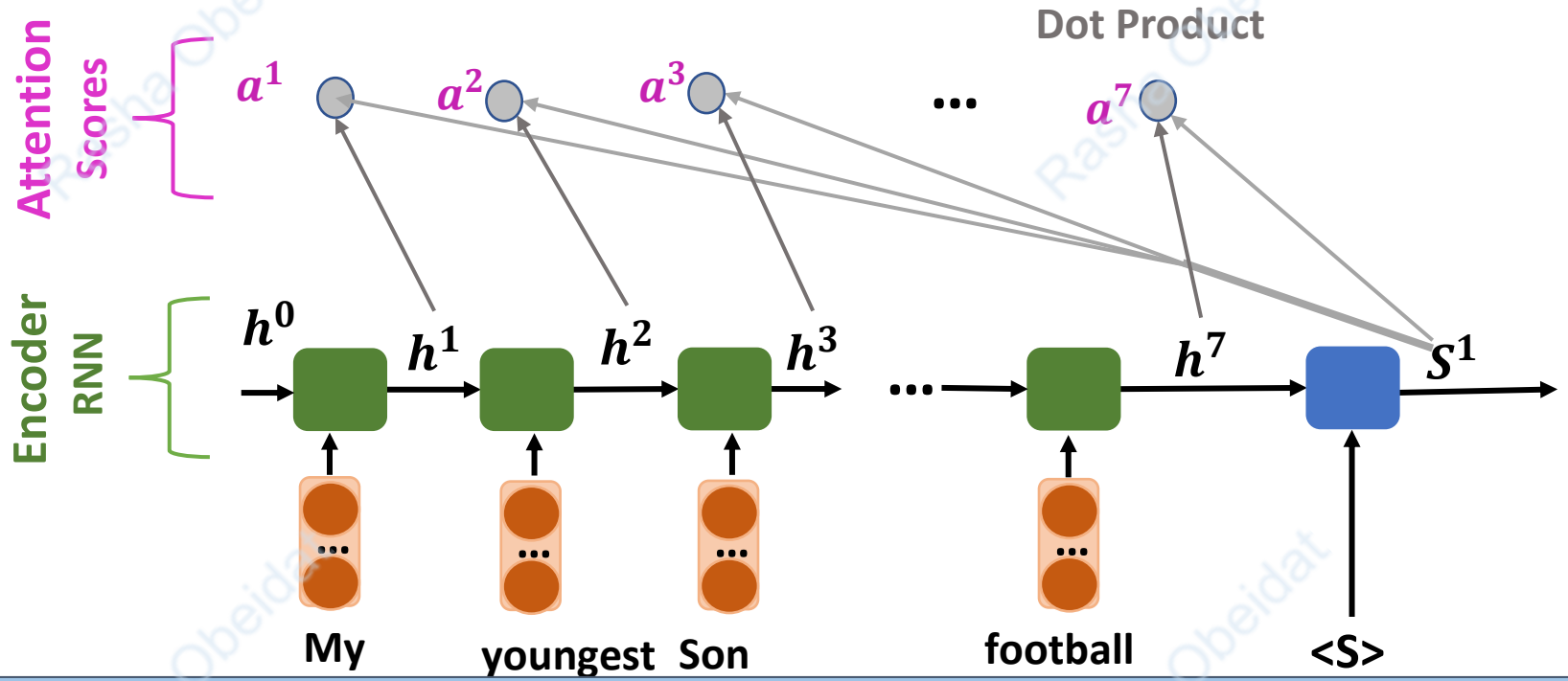
- Compute $a^{j,i} = f\left(h^i, \mathrm{S}^{\mathrm{j}}\right)$
- $h^i \in \mathbb{R}^{\mathrm{h}}, S^j \in \mathbb{R}^{\mathrm{h}}$



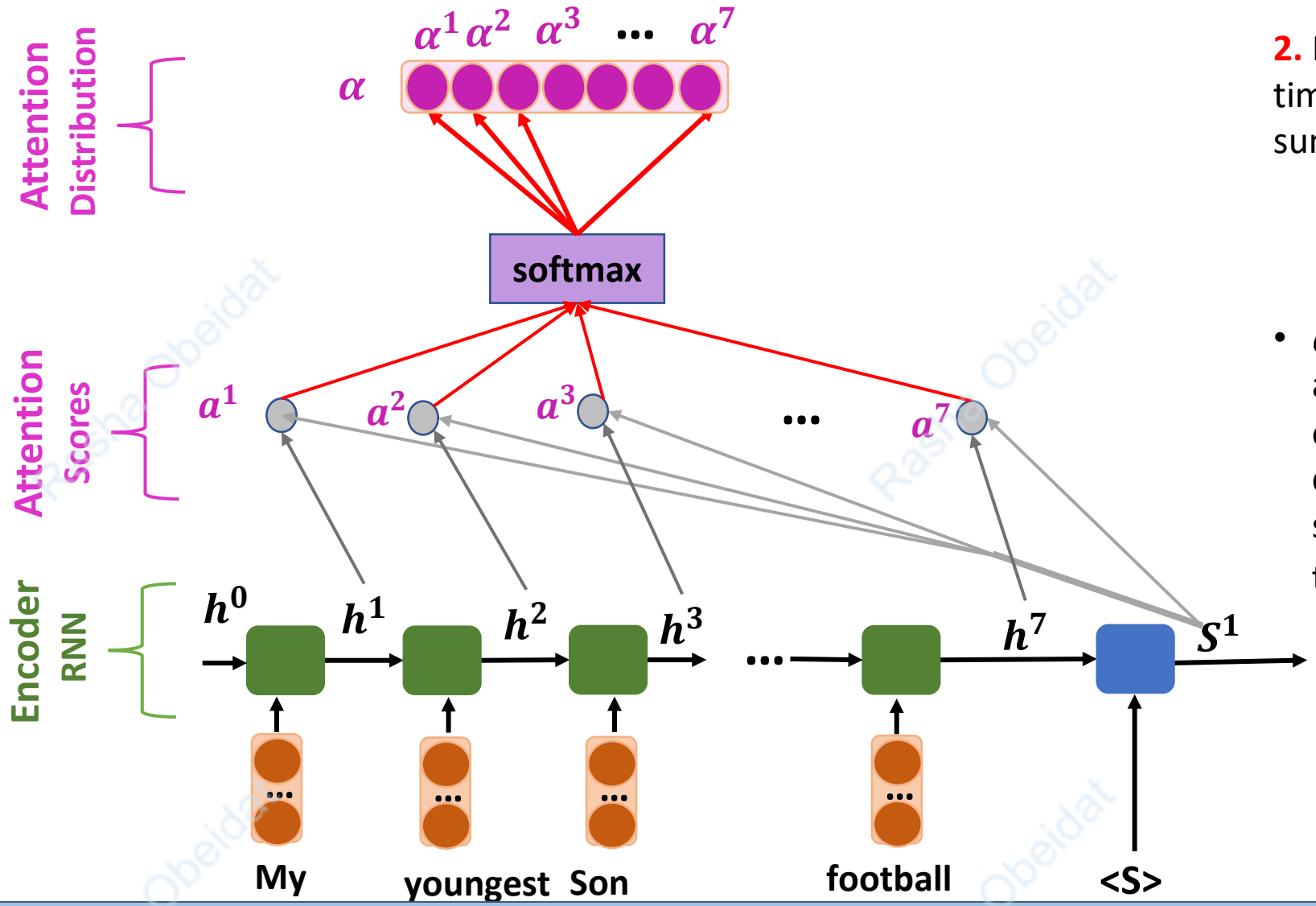
Attention

1. At each time step j in the decoding stage , for all steps of the encoder $i = 1, 2, \dots, m$

- Compute $a^{j,i} = f(h^i, S^j)$
- $h^i \in \mathbb{R}^h, S^j \in \mathbb{R}^h$



Attention



1. At each time step j in the decoding stage , for all steps of the encoder $i = 1, 2, \dots, m$

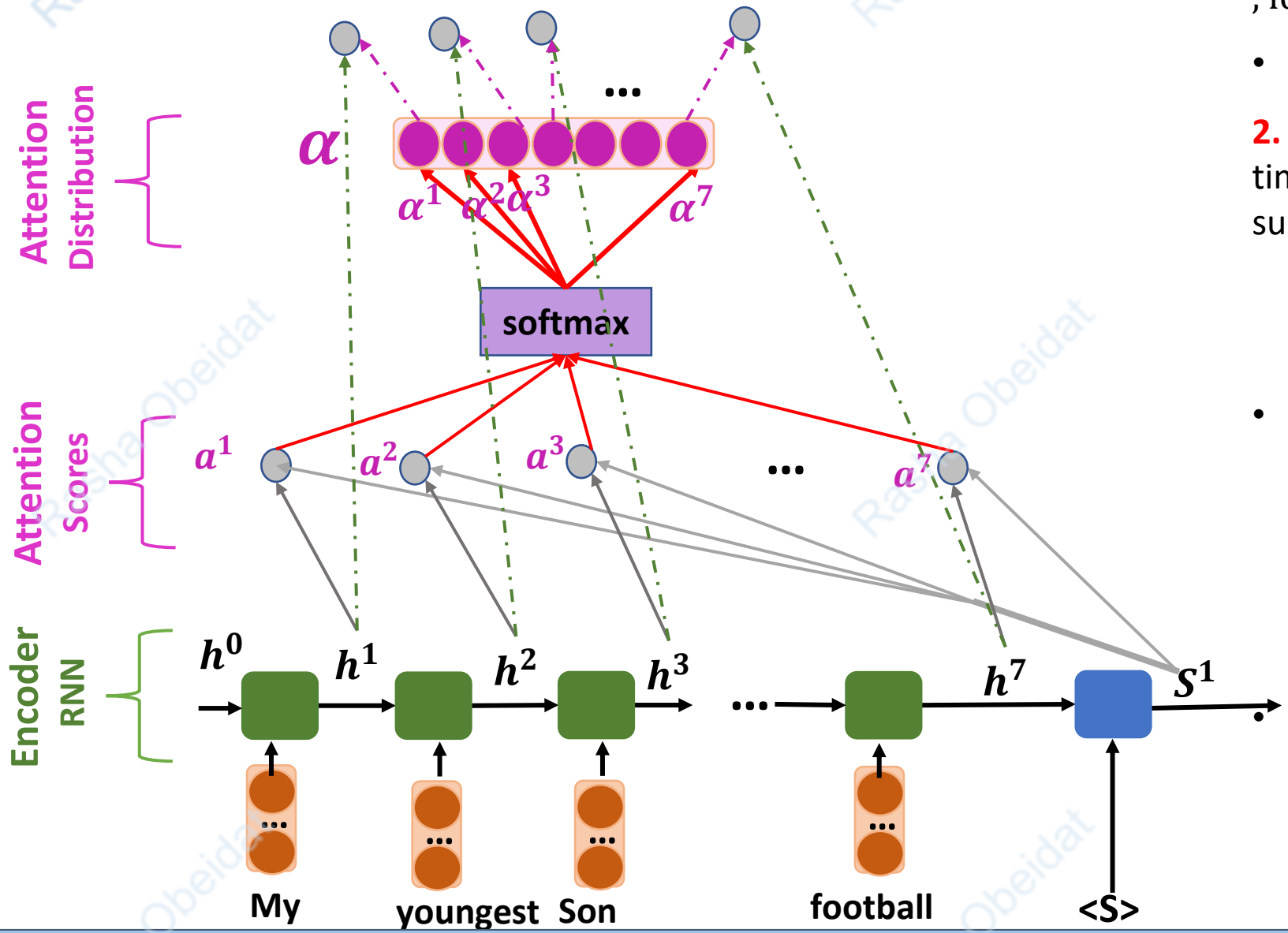
- Compute $a^{j,i} = f(h^i, S^j)$

2. Normalize the values of $a^{i,j}$ at the decoder time step j by using Softmax so they can sum up to one:

$$\alpha^{j,i} = \frac{\exp(a^{j,i})}{\sum_{k=1}^{m} \exp(a^{j,k})}$$

- $\alpha^j = [\alpha^{j,1}, \alpha^{j,2}, \dots, \alpha^{j,m}]$ is considered as attention distribution at time step j. In other words, $\alpha^{j,1}, \alpha^{j,2}, \dots, \alpha^{j,m}$ are considered as weights of the hidden states of the encoder when computing the context vector $\boldsymbol{c^j}$.

Attention



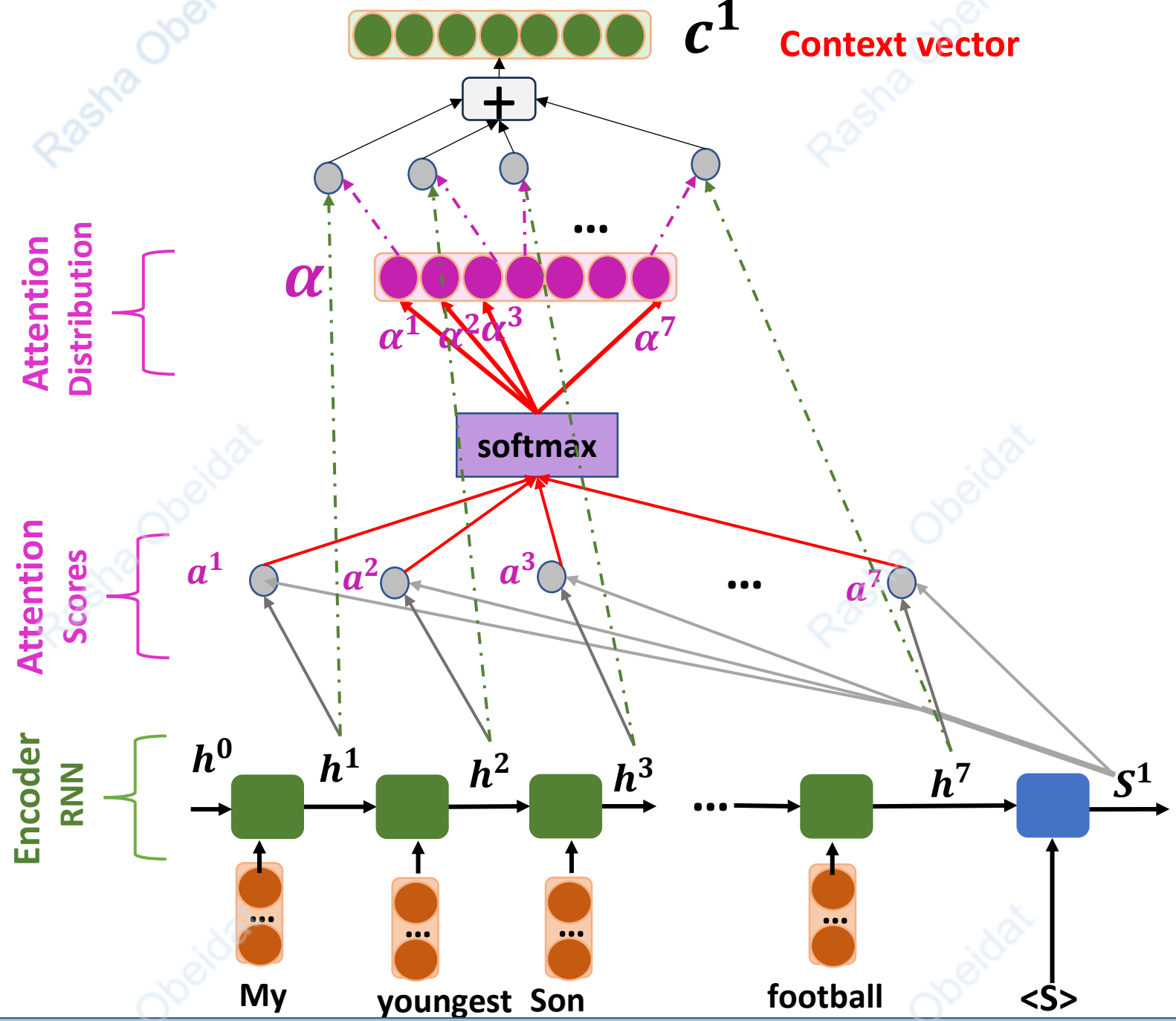
1. At each time step j in the decoding stage , for all steps of the encoder $i = 1, 2, \dots, m$

- Compute $a^{j,i} = f(h^i, S^j)$

2. Normalize the values of $a^{i,j}$ at the decoder time step j by using Softmax so they can sum up to one:

$$\alpha^{j,i} = \frac{\exp(a^{j,i})}{\sum_{k=1}^{m} \exp(a^{j,k})}$$

- $\alpha^j = [\alpha^{j,1}, \alpha^{j,2}, \dots, \alpha^{j,m}]$ is considered as attention distribution at time step j. In other words, $\alpha^{j,1}, \alpha^{j,2}, \dots, \alpha^{j,m}$ are considered as weights of the hidden states of the encoder when computing the context vector $\boldsymbol{c^j}$.
- Each hidden state h^i in the encoder is multiplied by its corresponding attention weight $\alpha^{j,i}$



1. At each time step j in the decoding stage , for all steps of the encoder $i = 1, 2, \dots, m$

- Compute $a^{j,i} = f(h^i, S^j)$

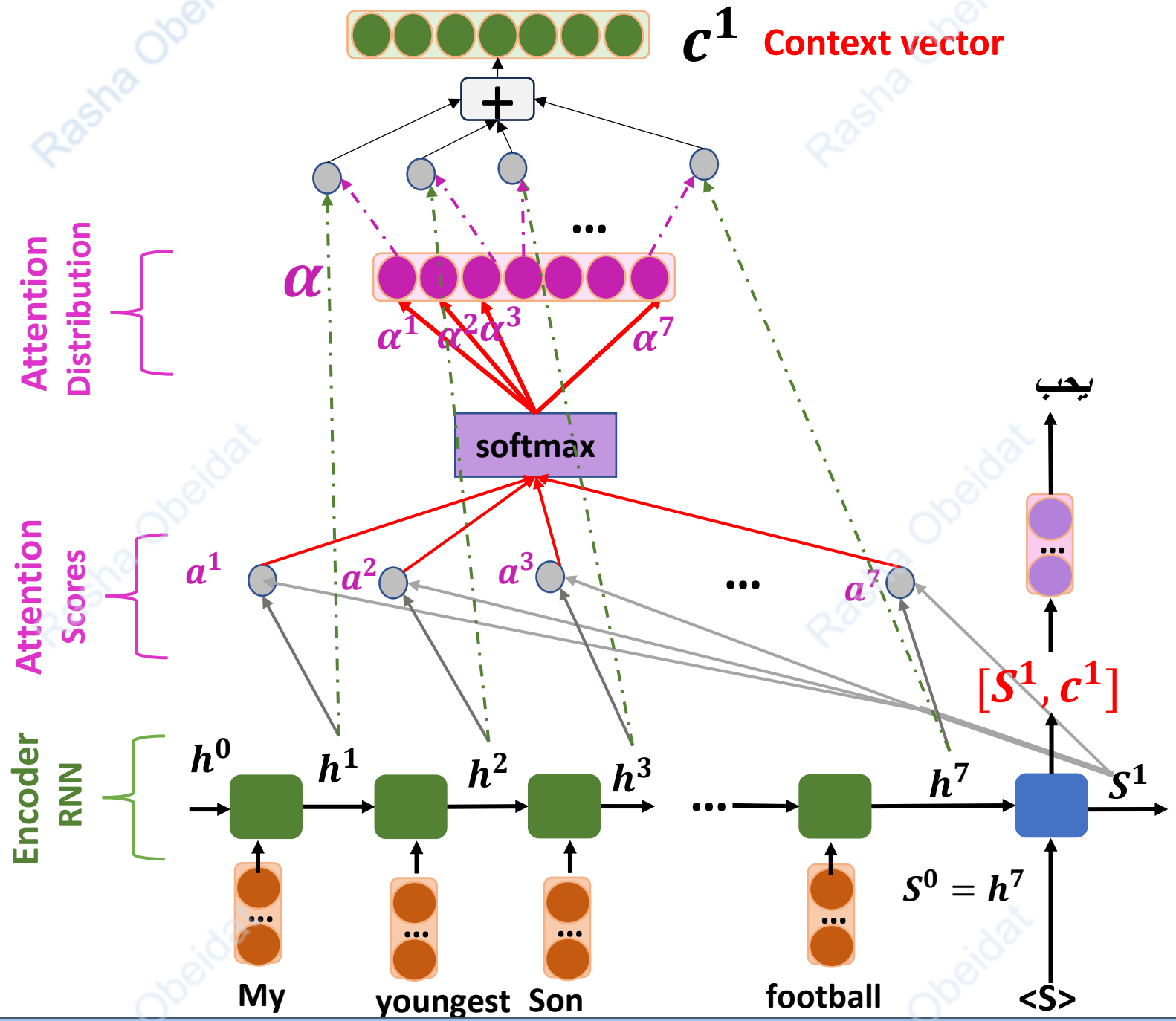
2. Normalize the values of $a^{i,j}$ at the decoder time step j by using Softmax so they can sum up to one:

$$\alpha^{j,i} = \frac{\exp(a^{j,i})}{\sum_{k=1}^{m} \exp(a^{j,k})}$$

- $\alpha^j = [\alpha^{j,1}, \alpha^{j,2}, \dots, \alpha^{j,m}]$ is considered as attention distribution at time step j. In other words, $\alpha^{j,1}, \alpha^{j,2}, \dots, \alpha^{j,m}$ are considered as weights of the hidden states of the encoder when computing the context vector $\boldsymbol{c^j}$.

3. The context vector $\boldsymbol{c^j} \in \mathbb{R}^{\boldsymbol{h}}$ is the weighted average of the hidden states of the encoder:

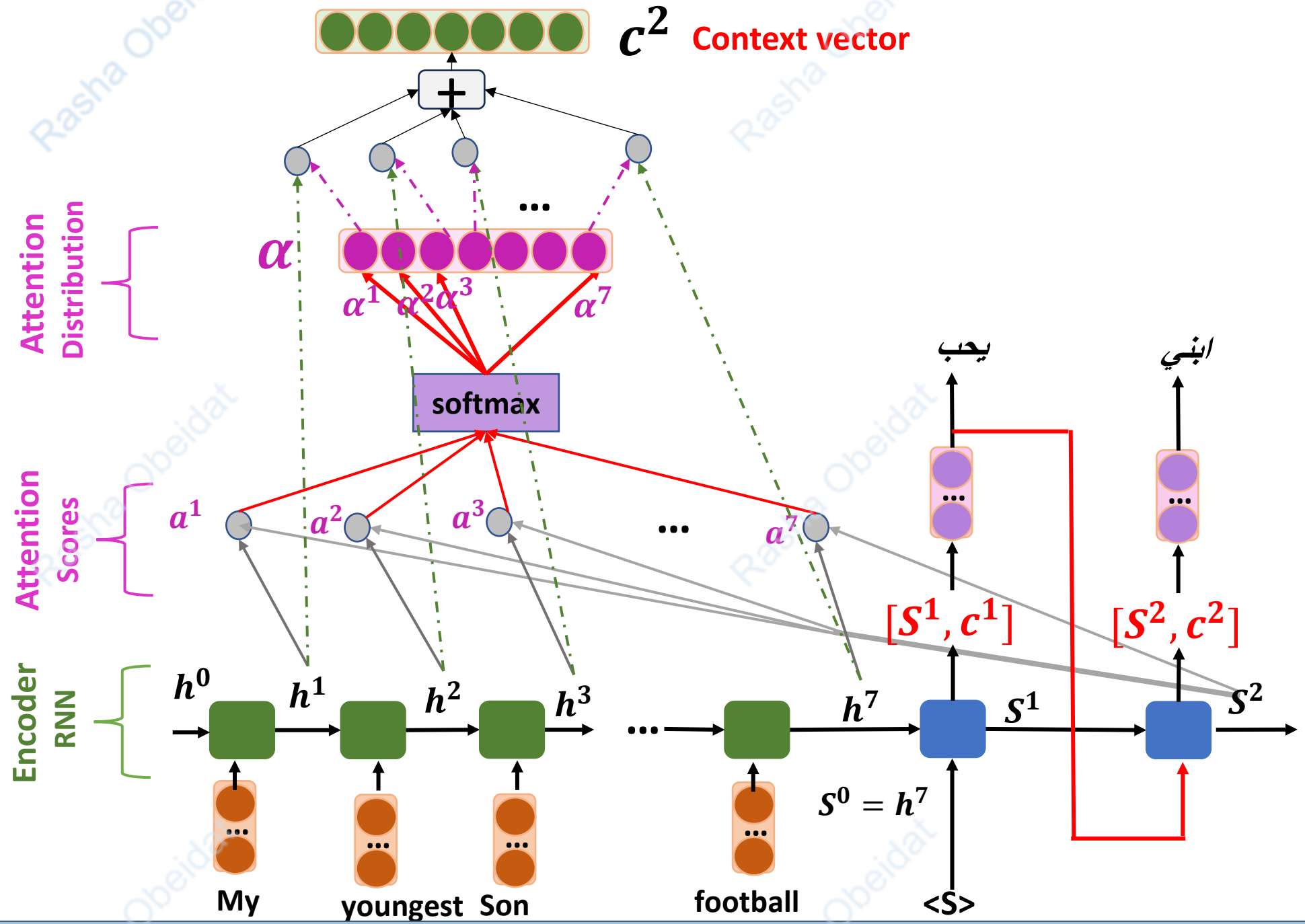
$$c^j = \sum_{i=1}^{m} \alpha^{j,i} h^i$$

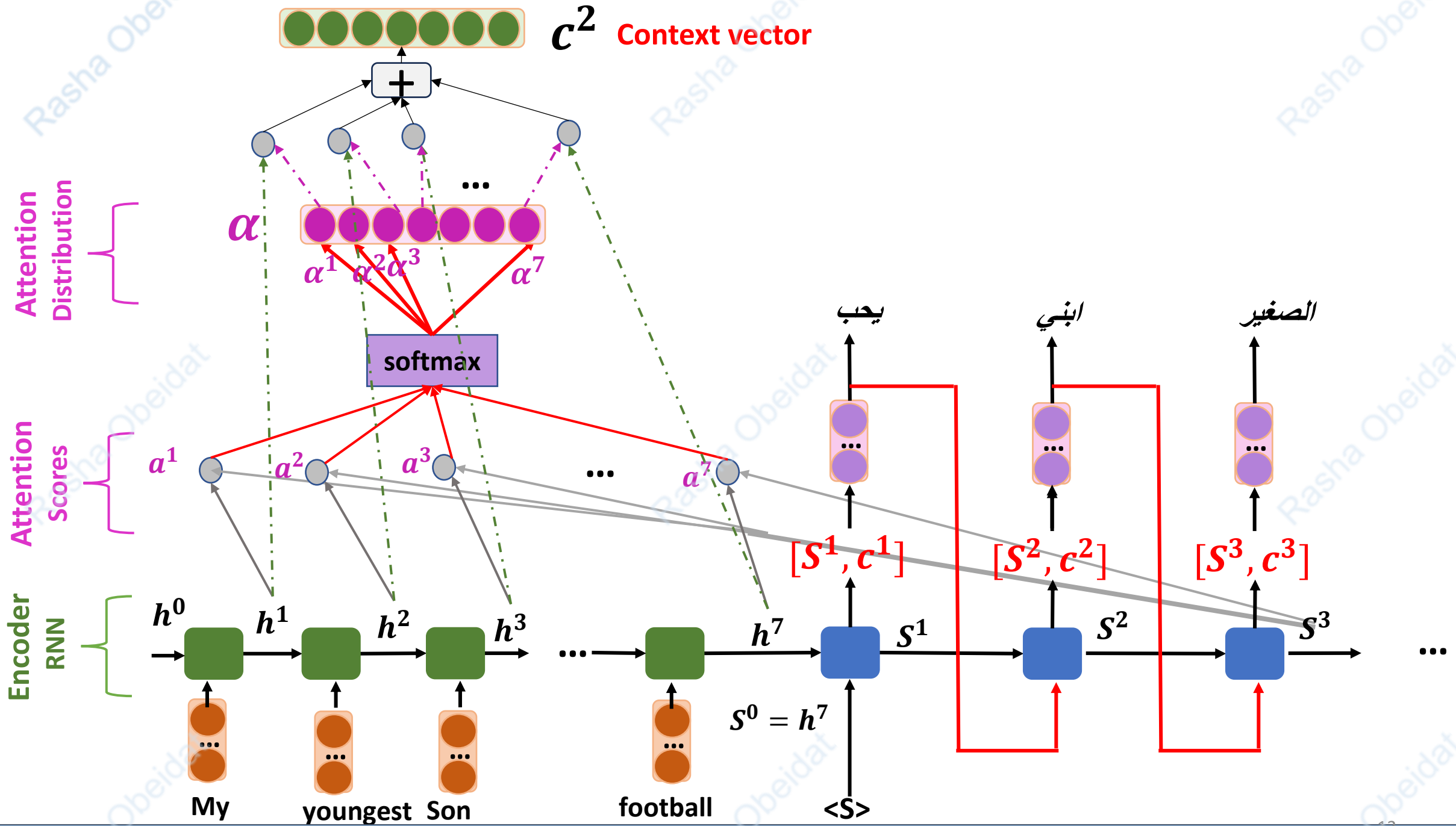


4. At each time step j in the decoding, A concatenation of S^j and c is used to compute the output of the current time step j

$$S^j = [S^j, c^j] \in \mathbb{R}^{2h}$$

and proceed as in the non-attention seq2seq model


c^2
Context vector
Attention Distribution
α
α^1 α^2 α^3 α^7
softmax
Attention Scores
a^1 a^2 a^3 ... a^7
Encoder RNN
h^0 h^1 h^2 h^3 ... h^7
$S^0 = h^7$
S^1 S^2
$[S^1, c^1]$ $[S^2, c^2]$
يحب
ابني
My youngest Son football <S>


c^2
Context vector
Attention Distribution
α
α^1 α^2 α^3 α^7
softmax
Attention Scores
a^1 a^2 a^3 ... a^7
Encoder RNN
h^0 h^1 h^2 h^3 ... h^7
$S^0 = h^7$
S^1 S^2 S^3 ...
$[S^1, c^1]$ $[S^2, c^2]$ $[S^3, c^3]$
يحب ابني الصغير
My youngest Son football <S>

Seq2Seq with attention for MT

Training Objective:

- Trained to minimize **cross-entropy loss**, which measures the difference between the predicted and true token probabilities.

Teacher Forcing:

- During training, the true target tokens are provided to the decoder to condition the next token prediction, ensuring stable and efficient learning.

Inference with Beam Search:

- During inference, **beam search** is often used to explore multiple possible translations and select the most likely one based on the model's predictions.

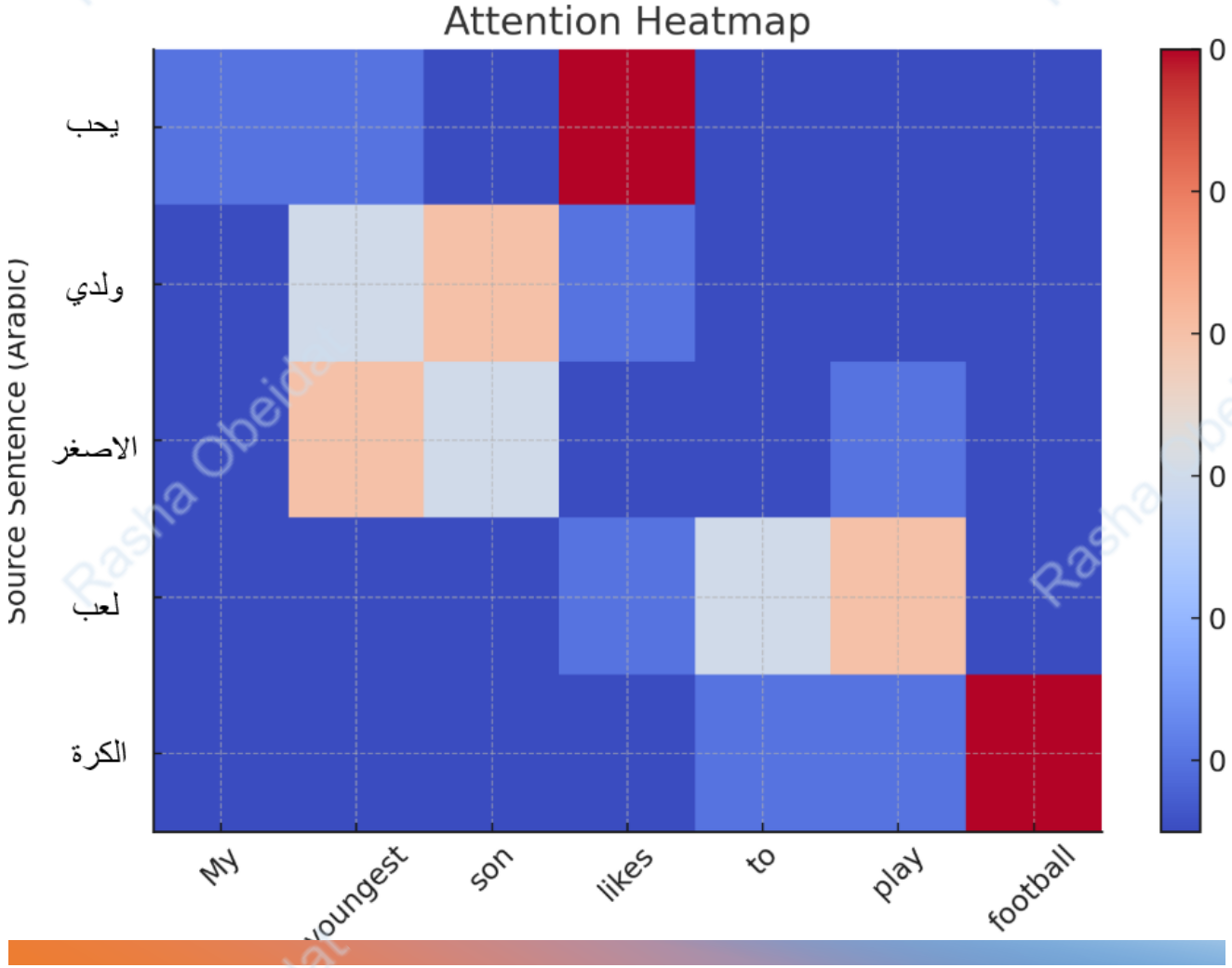
Why Attention is Powerful for MT

- **Improves NMT Performance**:
  - Enables the decoder to focus on specific, relevant parts of the source sentence.
  - Avoids information loss by dynamically attending to the input.
- **Solves the Bottleneck Problem**:
  - Instead of relying on a fixed-length vector, attention allows the decoder to directly look at the source sequence at each step.
  - Mitigates limitations of traditional Seq2Seq models.
- **Handles Vanishing Gradients:**
  - Provides a shortcut for gradients to flow back to faraway timesteps, making training easier.
  - Reduces dependency on long-term memory in RNNs.

Dr. Rasha Obeidat®

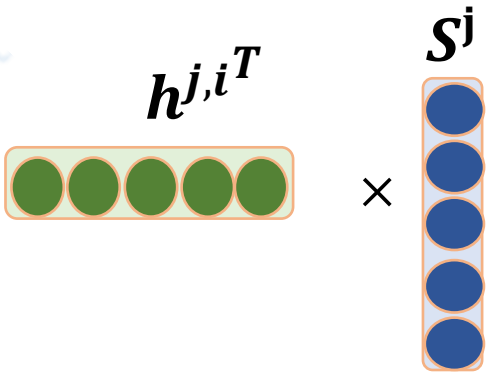
Why Attention is Powerful for MT

- **Enables Interpretability:**
  - Visualizing attention weights (***Attention heatmap***) shows which input tokens the model is focusing on at each decoding step.
- **Offers alignment for free:**
  - The model inherently learns alignment without explicit training for it.
  - Example: "يحب ولدي الاصغر لعب الكرة" → “My youngest son likes to play football " (heatmap visualization of alignments).



Popular Attention Scoring functions

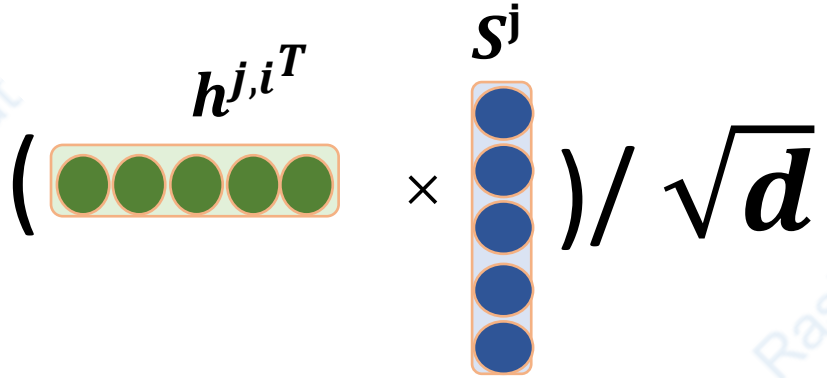
Dot Product Attention



$$f\left(h^{j,i}, S^j\right) = h^{j,i^T} S^j$$

No Need to learn Attention Weight Matrix

Scaled Dot Product Attention

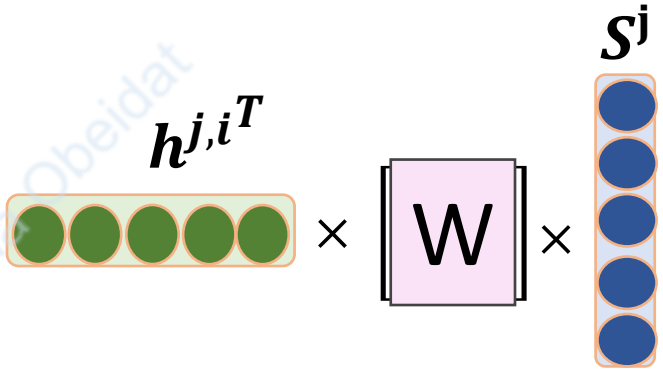


$$f\left(h^{j,i}, S^j\right) = \frac{h^{j,i^T} S^j}{\sqrt{d}}$$

Similar to the Dot-Product, but scaled by the square root of the dimension d to prevent the scores from growing too large.

Popular Attention Scoring functions

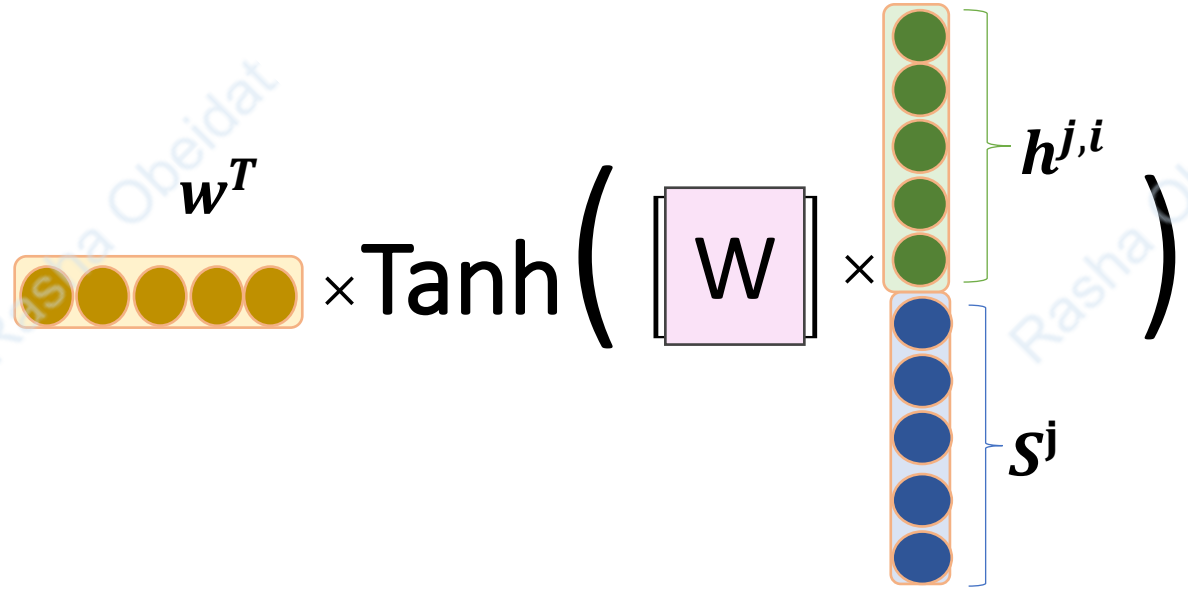
Bilinear Attention



$$f\left(h^{j,i}, S^j\right) = h^{j,i^T} W S^j$$

Also known as ***Multiplicative*** Attention

Multilinear Perceptron Attention



$$f\left(h^{j,i}, S^j\right) = \mathbf{w}^T \,.\, \tanh(\mathbf{W}[\mathbf{h}^{j,i}, \mathbf{S}^j])$$

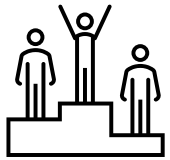
Also Known as ***additive*** Attention

Is attention only useful in seq2seq models?

- **The answer is no!**

- Attention can also be used to model dependencies **between inputs**, such as in Natural Language Inference (NLI), where we align and compare a **premise** and a **hypothesis**.

Let us compare two models for NLI:



- One that uses **attention**
- One that does **not**.

Natural Language Inference

- **Natural Language Inference** (NLI)
  - also known as **Textual Entailment (TE)**
  - The task of task of determining the **logical relationship** between two textual inputs: a **premise** and a **hypothesis.**
- **Types:**

2-Way NLI (Binary Classification):

- **Entailment**: The hypothesis logically follows from the premise.
- **Not Entailment**: The hypothesis does not follow (includes both contradiction and neutral cases).

3-Way NLI (Multi-Class Classification):

- **Entailment**: The hypothesis must be true if the premise is true.
- **Contradiction**: The hypothesis must be false if the premise is true.
- **Neutral**: The hypothesis is plausible but not guaranteed to be true or false based on the premise.

Natural Language Inference

- **Examples**
  - **Premise:** يلعب الولد كرة القدم في الحديقة.
  - **Hypothesis :** الطفل يمارس رياضة في الخارج.
  - Class: Entailment

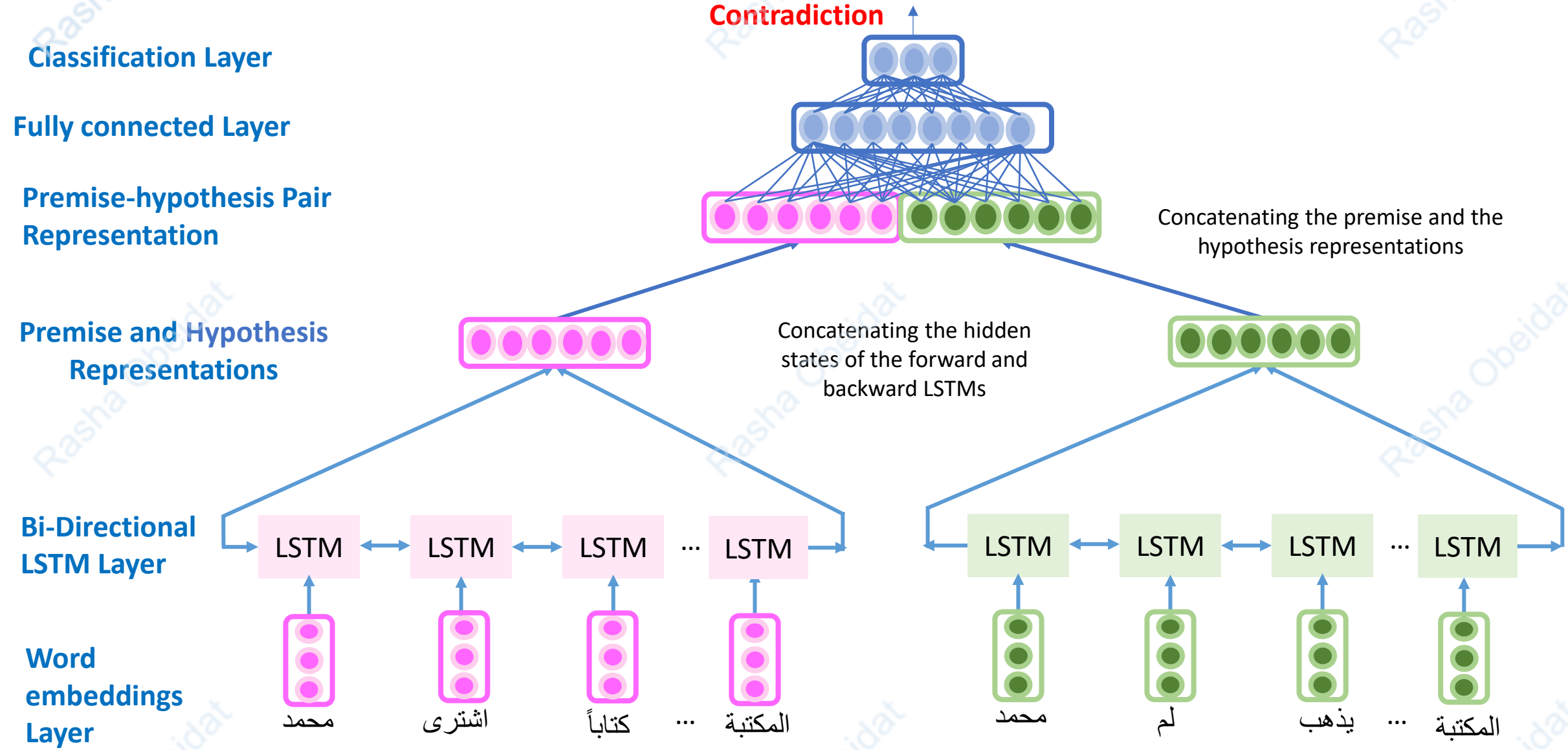
  - **Premise:** يلعب الولد كرة القدم في الحديقة.
  - **Hypothesis :** الولد يدرس فى المكتبة.
  - Class: Contradiction

  - **Premise:** يلعب الولد كرة القدم في الحديقة.
  - **Hypothesis :** يرتدي الولد قميصًا أحمر.
  - Class: Neutral

First Architecture

Rasha Obeidat

Bi-directional LSTM for NLI (no attention is used)



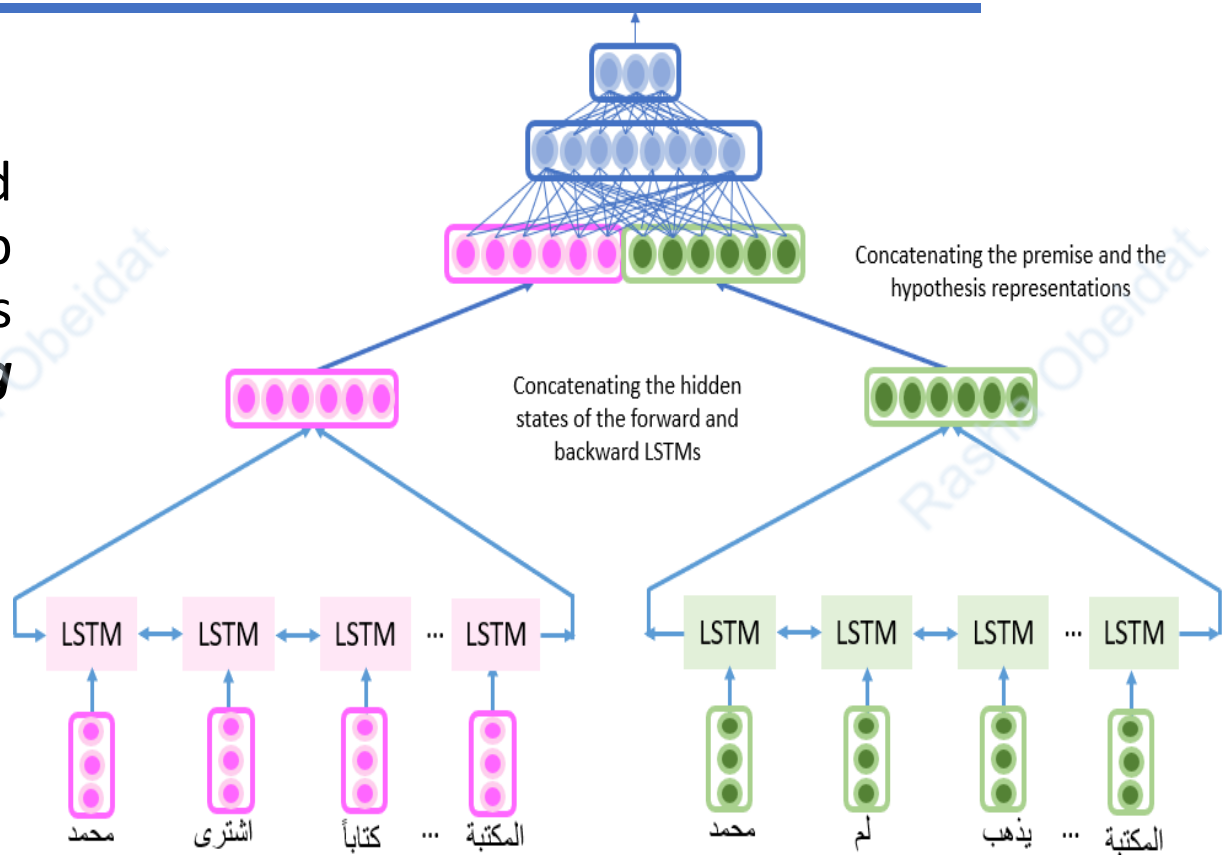
Bi-directional LSTM for NLI

Model Architecture:

- This model applies a BiLSTM-based architecture to model the relationship between a premise and a hypothesis sentence in the NLI task ***(without using attention mechanisms)***.

Limitation:

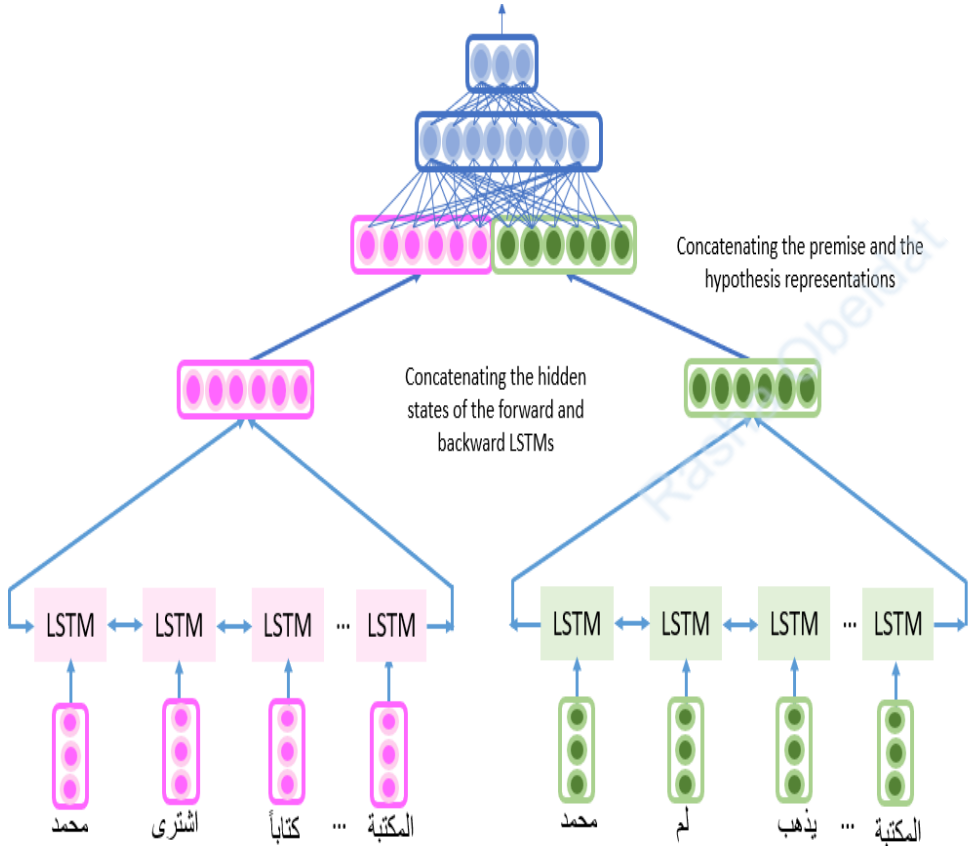
- This architecture does not use attention, so it treats both sentences as independent units before merging, which may lead to weaker alignment compared to attention-based models (like ESIM or DR-BiLSTM).



Bi-directional LSTM for NLI (Architecture Components)

Architecture Components:

1. **Word Embeddings Layer (bottom):** Both the premise and hypothesis sentences are tokenized and mapped into dense vector embeddings.
2. **Bi-Directional LSTM Layer:** Each sentence is processed separately by a BiLSTM, which captures both forward and backward contextual information for each word.
3. **Sentence Representation Layer:** A fixed-length representation of each sentence is obtained by concatenating the final forward and backward hidden states.
4. **Pair Representation:** The resulting vectors from the premise and hypothesis are concatenated to form a joint representation of the sentence pair.
5. **Fully Connected Layer & Classification:** The pair representation is passed through a fully connected layer, followed by a classification layer that outputs the relationship: ***Entailment***, ***Contradiction***, or ***Neutral***.



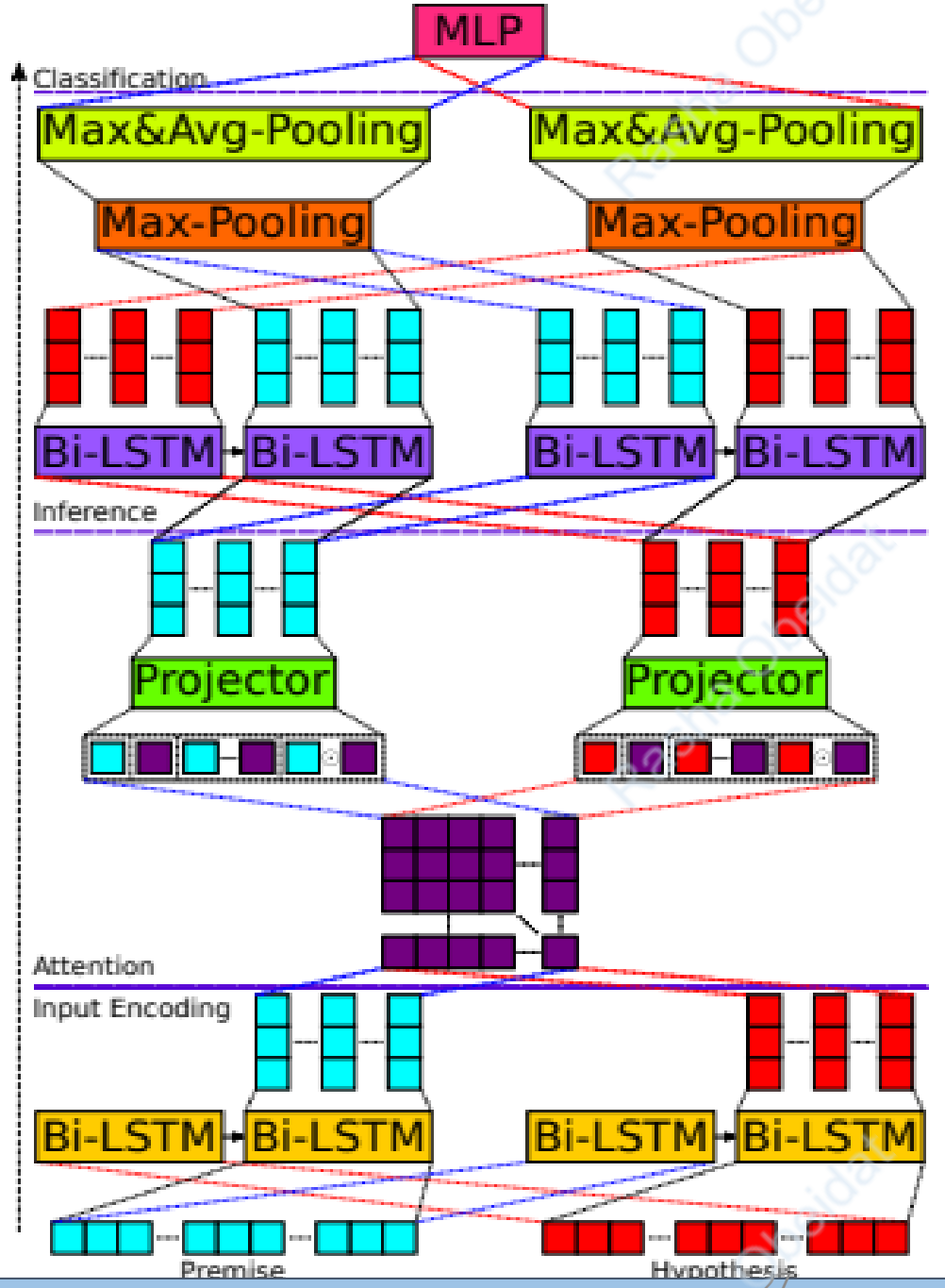
Second Architecture

Rasha Obeidat

Attention for Textual Entailment

Model Architecture:

- Uses **attention and dependent reading** to model the relationship between a **premise** and a **hypothesis** Instead of processing the two sentences independently
- It allows **each sentence to be encoded with awareness of the other**, using **attention to align** and compare their tokens.
- These aligned representations are then passed through inference and pooling layers before final classification.



*Ghaeini, Reza, et al. "**DR-BiLSTM: Dependent Reading Bidirectional LSTM for Natural Language Inference.**" Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers). 2018.*

Performance Comparison

- **Dataset**: SNLI (Stanford Natural Language Inference)
  - a widely-used 3-way textual entailment dataset developed by Stanford University.

- **Accuracy** of BI-LSTM (First Architecture) : 80.6%
- **Accuracy** of DR-BiLSTM (Second Architecture) : 88.5%

Dr. Rasha Obeidat®

Resources for Arabic NLI

- **ArEntail**:
  - Manually curated Arabic NLI dataset comprising **6,000 sentence**.
  - These pairs are sourced from **news headlines** and are annotated into three categories: ***Entailment***, ***Not-Entailment***
  - *Obeidat, Rasha, Yara Al-Harahsheh, Mahmoud Al-Ayyoub, and Maram Gharaibeh. "ArEntail: manually-curated Arabic natural language inference dataset from news headlines." Language Resources and Evaluation (2024): 1-27.*
- **ArbTE** :
  - Semi-automatically synthesized dataset.
  - Comprises only 618 examples.
  - *Alabbas, Maytham. "ArbTE: Arabic textual entailment." Proceedings of the Second Student Research Workshop associated with RANLP 2011. 2011.*

- These slides are the intellectual property of **Dr. Rasha Obeidat®**. If you wish to use them for teaching **your course**, please contact the author for permission.
  The slides may **not be modified or redistributed** without explicit permission.
- **Contact:**
  - Obeidat.rm@gmail.com
  - Rmobeidat@just.edu.jo



Rasha Obeidat